

ESOF 322

Software Engineering

LECTURE: 12

J. L. Pach

OUTLINE

- Software architecture

1. MVC
2. Layered architecture
3. Repository
4. Client–Server
5. Pipe and Filter

What every Scrum developer needs to know

BASED ON SOMMERVILLE, *SOFTWARE ENGINEERING*, CHAPTER 6

What is software architecture?

> A description of **how a software system is organized**: its main components and the relationships between them.

- Output of architectural design = a **model**: components + communication
- It is the **bridge between requirements and detailed design**
- Usually a simple **block diagram**: boxes = components, arrows = data / control flow



Architecture ≠ design of a function

Level	Question	Example
Code	How does this function work?	`ApplyGravity()`
Design	How do these classes/modules collaborate?	`Player` uses `World`
Architecture	What are the big parts and the rules between them?	Input → Update → Render; layers

> Architecture = the decisions that are **hard to change later**.

Agile and architecture: the myth

> **Myth:** "We are agile, so architecture will just emerge."

Sommerville:

- An early stage of an agile project should focus on an **overall architecture**
- *Incremental development of architectures is not usually successful*
- Refactoring **components** → relatively easy
- Refactoring the **architecture** → expensive (most components must change)

Cost of change

What changes	Example	Cost
A line / function	rename a variable	minutes
A component	rewrite the collision code	hours–days
The architecture	move from single file to layers / client–server	**weeks, touches everything**

> Scrum lets you change **direction** every sprint.
> Architecture decides **how expensive** that change is.

Good architecture = **keeps the cost of change low**.

Agile reality: *just enough* architecture

Not Big Design Up Front. Not zero design either.

- **Sprint 0 / first sprint**: sketch a conceptual architecture on a whiteboard
- Build a **walking skeleton**: thinnest end-to-end slice through all components
- Use **spikes** to test risky architectural decisions
- Postpone decisions to the **last responsible moment**, but *make them consciously*
- Record important decisions (**ADR** – Architecture Decision Record)

Architecture in the small vs. in the large

	Architecture in the small	Architecture in the large
Scope	one **program**	**enterprise** systems of many programs
Question	how is a program split into components?	how do systems / servers / companies interoperate?
Examples	our game, a mobile app	banking, e-commerce, hospital IT

This lecture: mostly **in the small**, which is where student teams work.

Why architecture matters

> Components implement **functional** requirements.
> The **architecture** dominates the **non-functional** characteristics. (Bosch)

Architecture affects:

- ⚒ **Performance**
- ⚒ **Robustness / reliability**
- ⚒ **Distributability**
- ⚒ **Maintainability**
- ⚒ **Security, safety, availability**

A study of *architecturally significant requirements* (Chen et al.) confirmed: it is the **non-functional requirements** that shape architecture most.

Three benefits of explicit architecture (Bass et al.)

1. **Stakeholder communication**

   A high-level picture everyone (PO, customer, devs) can discuss

2. **System analysis**

   Forces early thinking: can we meet performance, reliability, maintainability?

3. **Large-scale reuse**

   Similar systems share similar architectures (product lines, frameworks)

Architectural design = a series of decisions

There is **no formula**. Architects answer questions like:

- Is there a **generic application architecture** to use as a template?
- How will the system be **distributed** across processors / machines?
- Which **architectural patterns** fit?
- What is the **fundamental structure**?
- How do we **decompose** components into sub-components?
- How is **control** organized (central vs. event-driven)?
- Which organization best delivers the **non-functional requirements**?
- How will we **document** it?

Non-functional requirement → architectural response

Requirement	Architectural response
Performance	few, **large** components; critical operations close together; replicate
Security	**layers**; critical assets in the innermost layer
Safety	safety-related code **co-located** in one/few components
Availability	**redundant** components; replace without stopping
Maintainability	**small, self-contained** components; separate producers from consumers; avoid shared data structures

Trade-offs: there is no free lunch

- ⚒ Large components → **fast**, but harder to change
- ⚒ Small components → **maintainable**, but more communication overhead
- ⚒ Security checks → safer, but **slower**

Typical solutions:

- ⚒ Use **different patterns for different parts** of the system
- ⚒ Make the trade-off **explicit** in a Product Backlog item or ADR

> Question for the team: *Which NFR is most important for our product?* Ask the Product Owner early.

Architectural views: one diagram is never enough

A diagram shows **one perspective**. Kruchen's **4+1** model:

View	Shows	Useful for
Logical	key abstractions (classes, objects)	relating requirements to the model
Process	runtime interacting processes	performance, availability
Development	how code is split among developers/teams	managers, programmers
Physical	hardware and deployment	systems engineers
+1 Scenarios	use cases linking the views	everyone

> Plus Hofmeister's **conceptual view**: abstract, for stakeholders. *Almost always the first one you draw.*

Documentation in an agile team

Sommerville's position:

- Detailed architectural documentation from all 4 views is **usually a waste**, except for critical systems
- Develop **only the views that help communication**
- Prefer **informal notation**: whiteboard, block diagrams
- UML is most useful for **detailed documentation** or model-driven development
- Architecture Description Languages (ADLs are used for formal, detailed descriptions)
- **Good enough:** a whiteboard photo in the repo (`docs/architecture.png`) plus a short README section
- **Better:** one **ADR** per important decision (context → decision → consequences)

Architectural patterns: the idea

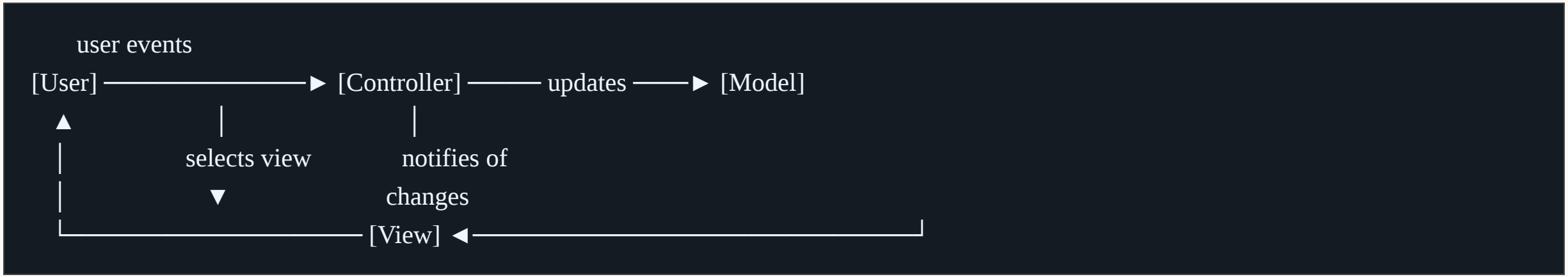
> A pattern is a **stylized description of good practice**, tried and tested in different environments.

- Describes: the **organization**, **when to use it**, **strengths and weaknesses**
- Not a blueprint: you **adapt** it
- Real systems usually **combine** several patterns

FIVE PATTERNS FOR TODAY:

1. MVC
2. Layered architecture
3. Repository
4. Client–Server
5. Pipe and Filter

Pattern 1: Model–View–Controller (MVC)



Model	data and business rules, knows nothing about the screen
View	how data is presented
Controller	handles input, updates the model, picks the view

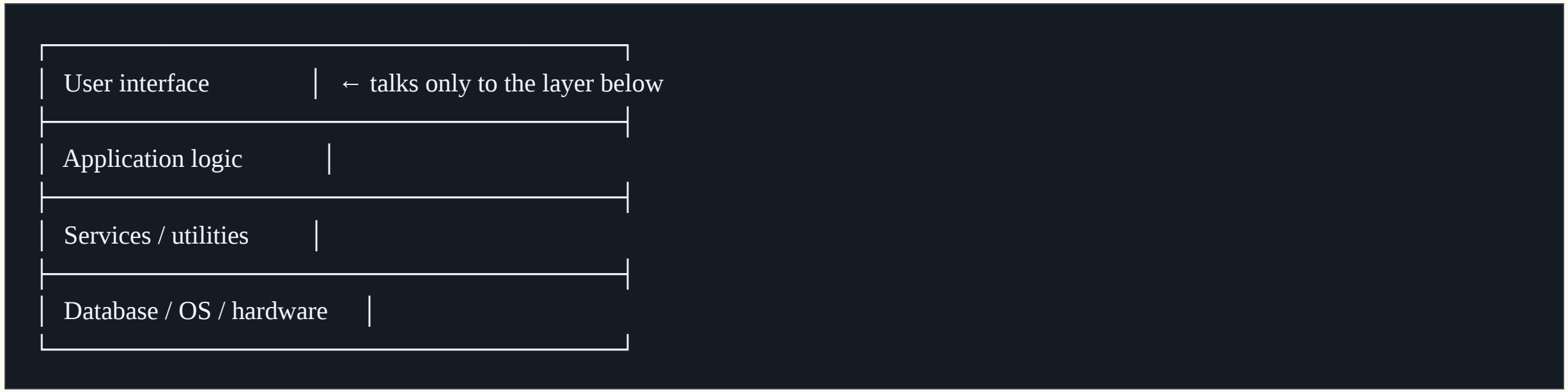
MVC: when to use it, pros and cons

Use when: there are **multiple ways to view and interact** with the same data, or future requirements for interaction are unknown.

Advantages	Disadvantages
Data can change independently of its representation (and vice versa)	Extra code and complexity when the model and interactions are simple
The same data can be shown in many views (e.g. table + chart)	Beginners often put everything into the Controller
A change in one view is reflected in all	

> Examples: web frameworks (Django, Rails, Spring MVC), GUI toolkits, mobile apps.

Pattern 2: Layered architecture



- Each layer provides **services to the layer above** and uses the layer below
- The **lower** layers are more general, the **upper** more specific
- Rule: **no skipping up**, lower layers never call upper ones

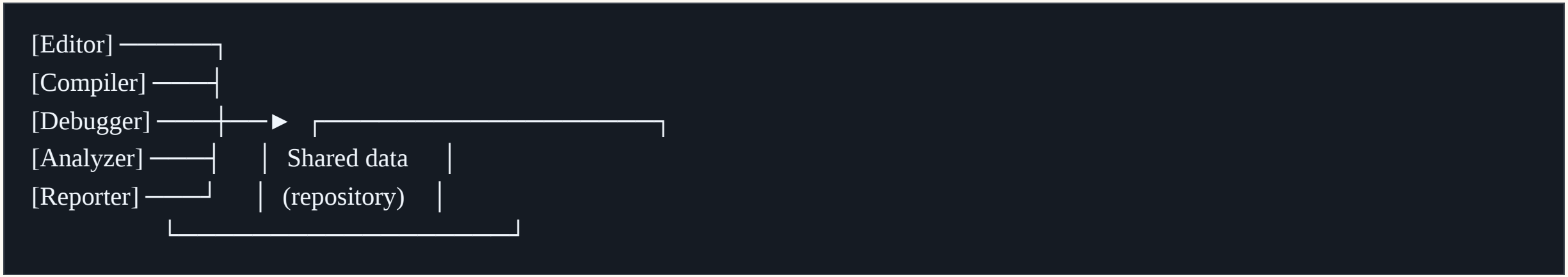
Layered: when to use it, pros and cons

Use when: building on top of existing systems, many teams each own one layer, or **security** requires a protected core.

Advantages	Disadvantages
A layer can be **replaced** if the interface stays the same	A clean separation is often hard to achieve
Supports **incremental development**	Performance suffers: a request travels through many layers
Portable: only the bottom layer is platform-specific	Layers may duplicate functionality (e.g. error handling)
Redundant features (e.g. authentication) can improve **dependability**	

Examples: OSI network model, operating systems, enterprise applications.

Pattern 3: Repository



- Components **do not talk to each other**; they read and write **shared data**
- Data is the integration point

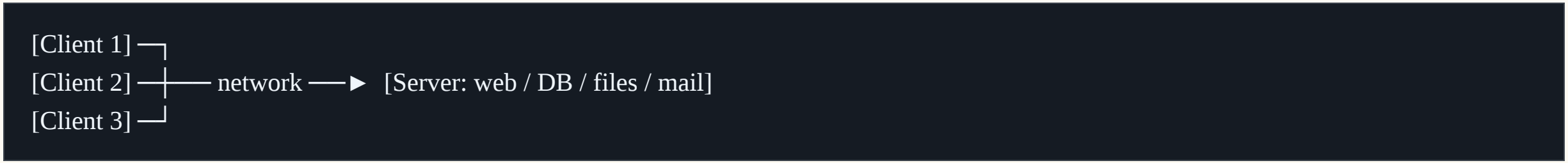
Use when: large amounts of data must be stored for a long time, or the system is **data-driven**.

Repository: pros and cons

Advantages	Disadvantages
Components are **independent**; they need not know about each other	The repository is a **single point of failure**
Changes by one component are visible to all	Everything goes through the repository: **inefficient**
Data is managed **consistently** (backup, security, access control)	**Distributing** the repository over several machines is difficult

Examples: IDEs, CASE tools, a shared database in a company information system, **a game's** `World` **state** used by many systems.

Pattern 4: Client–Server



- Functionality is organized into **services**, each delivered by a **server**
- **Clients** use the services; they know the server, the server does not need to know the clients

Use when: data in a shared database must be accessed from many locations, or the load **varies** and servers can be replicated.

Client–Server: pros and cons

Advantages	Disadvantages
Servers can be **distributed** across a network	Each server is a **single point of failure** (DoS, outage)
General functionality (e.g. printing) is available to all	**Performance is unpredictable**: it depends on network and load
Easy to **add servers** or upgrade them	Management problems if servers belong to different organizations

Examples: the web, online multiplayer games, mobile app + backend, e-mail.

Pattern 5: Pipe and Filter

```
[Input] ─► (Filter 1) ─► (Filter 2) ─► (Filter 3) ─► [Output]
           parse         transform      format
```

- Each **filter** transforms data; each **pipe** carries data to the next one
- Filters are independent and know nothing about their neighbours

Use when: data is processed in **stages**, typically batch processing (e.g. reports, compilers, ETL, image processing).

Pipe and Filter: pros and cons

Advantages	Disadvantages
Easy to understand	A **common data format** must be agreed on
Filters are **reusable**	Each filter parses its input again: **overhead**
Evolve by **adding** a filter	Not suitable for **interactive** systems
Can run **sequentially or concurrently**	

Examples: Unix shell (`cat | grep | sort`), compilers, rendering pipelines, CI pipelines.

Choosing a pattern: cheat sheet

If the main concern is...	Consider
Many views of the same data	**MVC**
Separation of concerns, replaceability, security	**Layered**
Many components working on shared, long-lived data	**Repository**
Many users, remote access, scaling	**Client–Server**
Data processed in stages	**Pipe and Filter**

Remember: **different parts** of a system may use **different patterns**.

Generic application architectures

Systems of the same type resemble each other, so you can start from a template.

Type	Idea	Examples
Transaction processing	user requests → database operations, must stay consistent	banking, e-commerce, booking
Information systems	layered access to a shared database	library, hospital, student records
Language processing	input → translated → executed (pipe & filter)	compiler, interpreter, SQL engine
Real-time / game loop	read input → update state → render, repeated every frame	video games, embedded controllers

Architecture in Scrum: where does it fit?

Scrum element	Architecture connection
Product Backlog	NFRs are backlog items too (performance, security)
Sprint 0 / Sprint 1	conceptual architecture + walking skeleton
Spike	time-boxed experiment to reduce architectural risk
Definition of Done	can include "respects the layer rules", "no new global state"
Retrospective	review: is the architecture slowing us down?
Refactoring	continuous small improvements inside the architecture

Who owns the architecture in Scrum?

- Scrum has **no "architect" role**
- The **whole team** is responsible for the design
- In practice, one or two experienced people **facilitate** the decisions
- Decisions are made **visibly** (whiteboard, ADR, pull request discussion)
- The **Product Owner** provides the NFR priorities; the team provides the technical options

> Danger: "architect" who decides alone → bottleneck.
> Danger: nobody decides → each developer builds his own architecture.

Case study: our Miner game

Our current `main.c`: one file, global variables, drawing and physics mixed together.

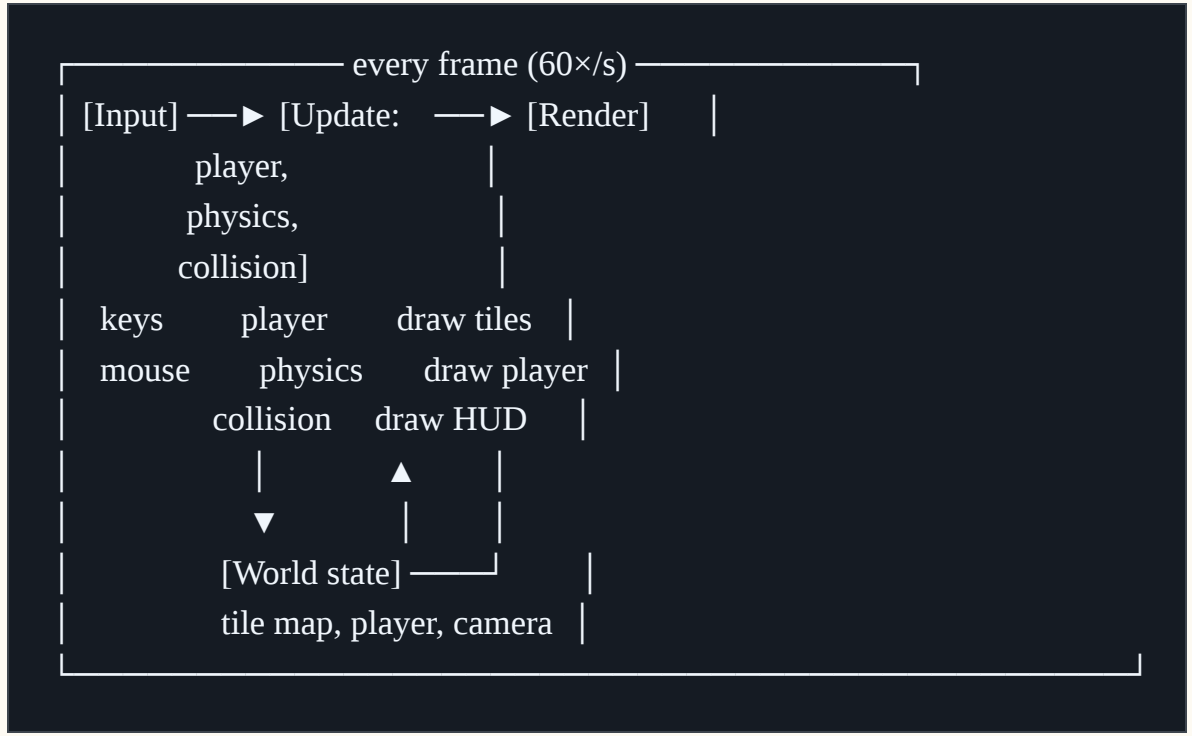


Everything depends on everything. Change one thing → another breaks.

This is sometimes called a **Big Ball of Mud**.

Case study: what architecture should it have?

A typical **game loop** architecture. The same three steps run every frame (60×/s):



- **Input** only reads the keyboard
- **Update** changes the world state
- **Render** only reads the world state and draws it

Which patterns do we recognize in the game?

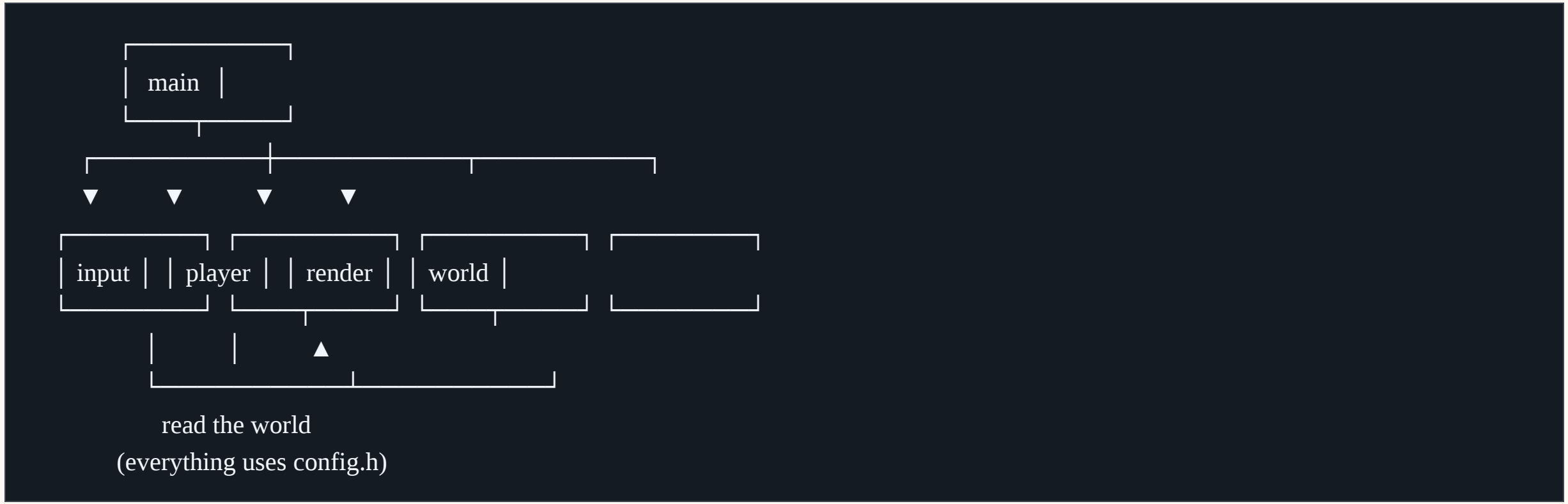
Part of the game	Pattern	Why
Main loop: Input → Update → Render	**Pipe and Filter** (per frame)	data flows through stages
`World` struct shared by many systems	**Repository**	one place for the shared state
Rules for Render, Player and Platform (raylib)	**Layered**	the game logic does not depend on drawing details
Separate data (`World`) from drawing	**MVC** (simplified)	the same data could be drawn as a minimap or a debug view
Multiplayer (future)	**Client–Server**	not needed now, and that is a conscious decision

> Real systems **combine** patterns. Naming them helps the team talk about the code.

Proposed module structure for Miner

```
???
```

Dependency diagram



Rule of thumb: arrows point in one direction only. If you need an arrow back, the design is wrong.

Non-functional requirements of Miner

NFR	Target	Architectural response
Performance	60 FPS, no frame longer than 16 ms	all logic in one thread, no per-frame allocations
Maintainability	a new student can add a feature in one week	small modules, one-way dependencies
Testability	collision can be tested without a window	`world` and `player` do not call raylib drawing
Portability	Windows now, maybe Linux later	raylib is the only platform dependency
Learnability	12 students work in parallel	few files, so fewer merge conflicts

Example ADR (Architecture Decision Record)

```
ADR-001: Separate world data from rendering

Status:   Accepted
Context:  main.c mixes tile data, physics and drawing.
          Every change in drawing risks breaking collision.
Decision: Tile map lives in world.c. render.c only reads it.
          No drawing calls inside player.c or world.c.
Consequences:
  + collision can be tested without opening a window
  + two students can work on render and physics in parallel
  - one more file and a few more function parameters
  - a short refactoring sprint now
```

One page, five minutes of writing. Future you will say thank you.

Typical architectural mistakes (anti-patterns)

Anti-pattern	Symptom	In our game
Big Ball of Mud	no visible structure, everything depends on everything	the original `main.c`
God object	one struct or file knows and does everything	one `Game` global with 40 fields
Golden Hammer	the same pattern for every problem	"everything must be MVC"
Speculative generality	abstractions for needs nobody has	an entity system for a game with one player
Architecture astronaut	diagrams without code	30 slides, no walking skeleton
Resume-driven design	technology chosen to look good	a microservice for the high-score table

Avoid both extremes: **no design** and **too much design**.

Architecture and code review

When you review a pull request, ask **architectural** questions, not only "does it compile":

- Does the change respect the **dependency rules**?
- Did a new global variable appear? Why?
- Does `render` now **modify** the game state?
- Is the new function in the **right module**?
- Will this make the next feature easier or harder?
- Is a decision worth an **ADR**?

> A review that only finds typos misses the most expensive mistakes.

Activity (15 min): sketch the architecture

In your team, at the whiteboard:

1. Draw a **conceptual view** of Miner as boxes and arrows (5 min)
2. Write the **top 3 non-functional requirements** and the response to each (5 min)
3. List **one decision** that would be expensive to change later (3 min)
4. Compare with the other team (2 min)

Rules:

- ⚒ No UML, only boxes and arrows
- ⚒ Every arrow needs a label: *what flows?* (data, events, calls)
- ⚒ Take a photo and commit it to `docs/architecture.png`

Key takeaways

1. Architecture = **how the system is organized**: components and their relationships
2. It is **expensive to change**, so it needs *some* attention early, even in Scrum
3. **Non-functional requirements** drive architectural decisions
4. Patterns (MVC, Layered, Repository, Client–Server, Pipe & Filter) are **vocabulary**, not blueprints
5. **Different parts** of a system can use **different patterns**
6. Document **just enough**: a whiteboard photo and a few ADRs
7. Architecture is **a team responsibility**, visible in code review

THANK

You